Modeling

Introduction

The data being used in this project is sourced from the Behavioral Risk Factor Surveillance System (BRFSS) which is a telephone based survey that is collected annually be the CDC. There are three data sets available from the BRFSS, but the scope of this analysis is limited to one, diabetes _ binary _ health _ indicators _ BRFSS2015.csv. The variables that will be used in EDA and modeling are BMI, whether or not the responder has higher cholesterol, and whether or not the responder consumes fruit 1 or more times per day.

The goal of this analysis is to train and evaluate three modeling approaches:

- Logistic Regression
- Classification Tree
- Random Forest

All models will be trained using 10-fold cross-validation and evaluated using log-loss on a holdout test set.

```
# Packages
library(tidymodels)
```

```
Warning: package 'tidymodels' was built under R version 4.3.3
```

```
-- Attaching packages ------------------------------------- tidymodels 1.3.0 --
```

```
v broom        1.0.7     v recipes      1.3.1
v dials        1.4.0     v rsample      1.3.0
v dplyr        1.1.4     v tibble       3.2.1
v ggplot2      3.5.2     v tidyr        1.3.1
v infer        1.0.9     v tune         1.3.0
v modeldata    1.4.0     v workflows    1.2.0
```

```
v parsnip      1.3.2     v workflowsets 1.1.1
v purrr        1.0.4     v yardstick    1.3.2

Warning: package 'broom' was built under R version 4.3.3

Warning: package 'dials' was built under R version 4.3.3

Warning: package 'scales' was built under R version 4.3.3

Warning: package 'dplyr' was built under R version 4.3.3

Warning: package 'modeldata' was built under R version 4.3.3

Warning: package 'purrr' was built under R version 4.3.3

Warning: package 'rsample' was built under R version 4.3.3

Warning: package 'tidyr' was built under R version 4.3.3

Warning: package 'tune' was built under R version 4.3.3

Warning: package 'workflows' was built under R version 4.3.3

Warning: package 'yardstick' was built under R version 4.3.3

-- Conflicts --------------------------------------- tidymodels_conflicts() --
x purrr::discard() masks scales::discard()
x dplyr::filter()  masks stats::filter()
x dplyr::lag()     masks stats::lag()
x recipes::step()  masks stats::step()
```

```r
library(yardstick)

# Read in data
data <- read.csv("diabetes_binary_health_indicators_BRFSS2015.csv", fileEncoding = "latin1")

data$HighChol <- as.factor(data$HighChol)
data$Fruits <- as.factor(data$Fruits)
data$Diabetes_binary <- as.factor(data$Diabetes_binary)

# Select relevant variables
data_model <- data |>
  select(Diabetes_binary, BMI, HighChol, Fruits)


# Split data
set.seed(111)
data_split <- initial_split(data_model, prop = 0.7)
data_train <- training(data_split)
data_test <- testing(data_split)
data_cv_folds <- vfold_cv(data_train, 10)
```

Logistic Regression

Logistic regression is utilized when the response variable of interest is binary, as seen in this data using Diabetes_binary. The logit link function models the probability that the response is in a particular category. In our case, this would be the probability that the responder does or does not have diabetes.

```r
# Logistic Regression Recipes
LR1_rec <- recipe(Diabetes_binary ~ BMI + HighChol + Fruits,
 data = data_train) |>
 step_normalize(BMI) |>
 step_dummy(HighChol) |>
 step_dummy(Fruits)

LR2_rec <- recipe(Diabetes_binary ~ BMI,
 data = data_train) |>
 step_normalize(BMI)

LR3_rec <- recipe(Diabetes_binary ~ BMI + HighChol,
 data = data_train) |>
```

```
 step_normalize(BMI) |>
 step_dummy(HighChol)

# Set engine
LR_spec <- logistic_reg() |>
 set_engine("glm")

# Set workflows
LR1_wkf <- workflow() |>
 add_recipe(LR1_rec) |>
 add_model(LR_spec)

LR2_wkf <- workflow() |>
 add_recipe(LR2_rec) |>
 add_model(LR_spec)

LR3_wkf <- workflow() |>
 add_recipe(LR3_rec) |>
 add_model(LR_spec)

# Fit to CV folds
LR1_fit <- LR1_wkf |>
 fit_resamples(data_cv_folds, metrics = metric_set(mn_log_loss))

LR2_fit <- LR2_wkf |>
 fit_resamples(data_cv_folds, metrics = metric_set(mn_log_loss))

LR3_fit <- LR3_wkf |>
 fit_resamples(data_cv_folds, metrics = metric_set(mn_log_loss))

# Collect metrics
bind_rows(
  collect_metrics(LR1_fit) |> mutate(model = "LR1")
    |> select(-.config, - .estimator, -n),
  collect_metrics(LR2_fit) |> mutate(model = "LR2")
    |> select(-.config, - .estimator, -n),
  collect_metrics(LR3_fit) |> mutate(model = "LR3")
    |> select(-.config, - .estimator, -n)
)
```

```
# A tibble: 3 x 4
  .metric      mean  std_err model
```

```
  <chr>       <dbl>    <dbl> <chr>
1 mn_log_loss 0.367 0.00130  LR1
2 mn_log_loss 0.383 0.000948 LR2
3 mn_log_loss 0.367 0.00131  LR3
```

It can be seen that the best fit model is LR1 which is a purely additive model using all 3 variables of interest. This model has the lowest log loss, therefore, it performs the best in classification.

Classification Tree

Tree based methods attempt to split up the predictor space into regions. In a classification tree, regions designate the most prevalent class as the prediction.

```
# Creating the tree recipe
tree_rec <- recipe(Diabetes_binary ~ ., data = data_train) |>
 step_normalize(BMI) |>
 step_dummy(HighChol) |>
 step_dummy(Fruits)

# Setting tree mode and tuning
tree_mod <- decision_tree(tree_depth = tune(),
                          min_n = 20,
                          cost_complexity = tune()) |>
  set_engine("rpart") |>
  set_mode("classification")

# Create workflow
tree_wkf <- workflow() |>
  add_recipe(tree_rec) |>
  add_model(tree_mod)

# Select tuning parameters using CV
tree_grid <- grid_regular(cost_complexity(),
                          tree_depth(),
                          levels = c(10, 3))

tree_fits <- tree_wkf |>
  tune_grid(resamples = data_cv_folds,
            grid = tree_grid,
             metrics = metric_set(mn_log_loss))
```

```
# Checking metrics
tree_fits |>
 collect_metrics() |>
 filter(.metric == "mn_log_loss") |>
 arrange(mean)
```

```
# A tibble: 30 x 8
   cost_complexity tree_depth .metric     .estimator  mean     n std_err .config
             <dbl>      <int> <chr>       <chr>      <dbl> <int>   <dbl> <chr>
 1    0.0000000001         15 mn_log_loss binary     0.377    10 0.00120 Prepro~
 2    0.000000001          15 mn_log_loss binary     0.377    10 0.00120 Prepro~
 3    0.00000001           15 mn_log_loss binary     0.377    10 0.00120 Prepro~
 4    0.0000001            15 mn_log_loss binary     0.377    10 0.00120 Prepro~
 5    0.000001             15 mn_log_loss binary     0.377    10 0.00120 Prepro~
 6    0.00001              15 mn_log_loss binary     0.378    10 0.00122 Prepro~
 7    0.0001                8 mn_log_loss binary     0.378    10 0.00124 Prepro~
 8    0.0000000001          8 mn_log_loss binary     0.378    10 0.00124 Prepro~
 9    0.000000001           8 mn_log_loss binary     0.378    10 0.00124 Prepro~
10    0.00000001            8 mn_log_loss binary     0.378    10 0.00124 Prepro~
# i 20 more rows
```

```
# Selecting best fit
tree_best_params <- select_best(tree_fits, metric = "mn_log_loss")

tree_final_wkf <- tree_wkf |>
  finalize_workflow(tree_best_params)

tree_final_fit <- tree_final_wkf |>
  last_fit(data_split, metrics = metric_set(mn_log_loss))

tree_final_fit |> collect_metrics()
```

```
# A tibble: 1 x 4
  .metric     .estimator .estimate .config
  <chr>       <chr>          <dbl> <chr>
1 mn_log_loss binary         0.379 Preprocessor1_Model1
```

Random Forest

Random forest modeling is an extension of classification trees in which multiple classification trees are generated using different subsets of the data and variables. The most common

classification across the trees becomes the prediction for that level.

```
# Create forest recipe
tree_rec <- recipe(Diabetes_binary ~ ., data = data_train) |>
 step_normalize(BMI) |>
 step_dummy(HighChol) |>
 step_dummy(Fruits)

# Set engine and mode
rf_spec <- rand_forest(mtry = tune()) |>
 set_engine("ranger") |>
 set_mode("classification")

# Create workflow
rf_wkf <- workflow() |>
 add_recipe(tree_rec) |>
 add_model(rf_spec)

# Fit to CV folds
f_fit <- rf_wkf |>
 tune_grid(resamples = data_cv_folds,
 grid = 7,
 metrics = metric_set(accuracy, mn_log_loss))
```

```
i Creating pre-processing data to finalize unknown parameter: mtry
```

```
Warning: package 'ranger' was built under R version 4.3.3
```

```
# Checking metrics
f_fit |>
 collect_metrics() |>
 filter(.metric == "mn_log_loss") |>
 arrange(mean)
```

```
# A tibble: 3 x 7
   mtry .metric     .estimator  mean     n std_err .config
  <int> <chr>       <chr>      <dbl> <int>   <dbl> <chr>
1     2 mn_log_loss binary     0.361    10 0.00133 Preprocessor1_Model2
2     3 mn_log_loss binary     0.363    10 0.00128 Preprocessor1_Model3
3     1 mn_log_loss binary     0.367    10 0.00118 Preprocessor1_Model1
```

```r
# Select best tuning parameter
rf_best_params <- select_best(f_fit, metric = "mn_log_loss")

# Refit on entire training with best tuning parameter
final_wkf <- rf_wkf |>
 finalize_workflow(rf_best_params)
rf_final_fit <- final_wkf |>
 last_fit(data_split, metrics = metric_set(accuracy, mn_log_loss))

rf_final_fit |> collect_metrics() |> filter(.metric == "mn_log_loss")
```

```
# A tibble: 1 x 4
  .metric     .estimator .estimate .config
  <chr>       <chr>          <dbl> <chr>
1 mn_log_loss binary         0.361 Preprocessor1_Model1
```

Final Model Selection

```r
lr_metrics <- collect_metrics(LR1_fit) |>
  filter(.metric == "mn_log_loss") |>
  mutate(model = "Logistic Regression") |>
  rename(.estimate = mean)

tree_metrics <- collect_metrics(tree_final_fit) |>
  filter(.metric == "mn_log_loss") |>
  mutate(model = "Classification Tree")

rf_metrics <- collect_metrics(rf_final_fit)|>
  filter(.metric == "mn_log_loss") |>
  mutate(model = "Random Forest")

final_results <- bind_rows(lr_metrics, tree_metrics, rf_metrics) |>
  select(model, .metric, .estimate)

print(final_results)
```

```
# A tibble: 3 x 3
  model               .metric     .estimate
  <chr>               <chr>           <dbl>
```

```
1 Logistic Regression mn_log_loss     0.367
2 Classification Tree mn_log_loss     0.379
3 Random Forest       mn_log_loss     0.361
```

It can be seen that the model that performed the best at predicting whether the responder has diabetes or not is the Random Forest model. The Random Forest model has the smallest log loss showing of 0.361, therefore, it is the best performing model.